

Learning Node Representations via Sketching the Generative Process With Events Benefits Link Prediction on Heterogeneous Networks

Xuan Guo , Pengfei Jiao , *Member, IEEE*, Danyang Shi , Jie Li, and Jinpeng Wang

Abstract—The Heterogeneous Information Network (HIN) stands out as a prominent tool for depicting interactions in real-world systems. Recently, representation learning on HINs has attracted significant attention, as the structured and compact output embeddings offer great convenience for network analysis and graph machine learning tasks. While existing HIN representation learning methods excel in supervised training or direct proximity reconstruction, yielding satisfactory performance in tasks like node clustering and classification, they often overlook the critical HIN generative process characterized by numerous events. As a result, these methods fail to preserve the higher-order interactions among the nodes and predict potential links in HINs. To address these limitations, we propose a Contrastive Learning method via Events on Heterogeneous Information Networks (CLEH). CLEH delineates the generative process from the local structure (nodes) to the higher-order structure (events) in HINs. We design a novel event-level contrastive learning procedure, endowing representations with the capability to capture higher-order relations among nodes. Moreover, CLEH leverages a normalizing flow model as the encoder to enhance the expressiveness of embeddings. Experimental results on HIN datasets demonstrate the significant superiority of CLEH in link prediction compared to popular baselines.

Index Terms—Heterogeneous network, network embedding, higher-order semantic, contrastive learning, link prediction.

I. INTRODUCTION

NETWORKS, a paradigm for modeling relations between different objects, have attracted considerable attention from researchers in different fields [1], [2], [3]. Real-world networks, in many instances, are formed through a series of **events** and constituted of multiple types of nodes and edges. For example, an academic network is shaped via the stream of publication events as shown in Fig. 1. Each publication is uniquely identified by the paper, referred to as the central node, and is associated with its authors, venue, and terms. It

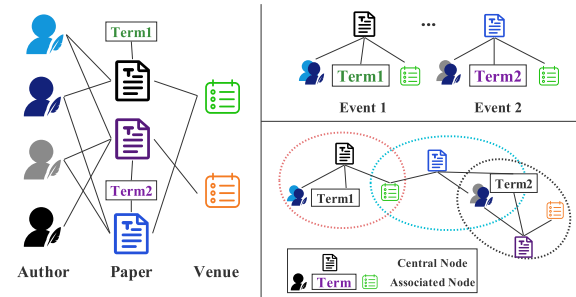


Fig. 1. Left: A HIN example, i.e., an academic network having four types of nodes including authors, papers, terms, and venues. Right: Depiction of some Events and their relations in the example. Each event contains one **Central Node** as the unique identifier for the event, along with several **Associated Nodes** connected to it. The number and type of shared nodes of two connected events may be different.

is their coupling relations that construct the network topology. To depict the various types of nodes and edges in the data, the **Heterogeneous Information Network (HIN)** [4], [5] has been proposed to explore many real-world scenarios and expressed strong representation capabilities across various mining tasks.

Nowadays, the unstructured data form and growing scale pose challenges for mining and analysis tasks on real-world HINs. Coping with the current situation, learning effective representations (or embeddings) of HINs has developed into a hot topic in recent years [6], [7], [8]. HIN representation learning aims to map the different types of nodes in a HIN into a latent vector space. With the learned embeddings, the topological structures and statistical properties of HINs can be effectively revealed and preserved. Subsequently, tasks on HINs, such as node classification and link prediction, can be addressed in the latent space with trained naive machine learning models. The classic HIN representation learning methods generally fall into three categories [6]: proximity-preserving [9], [10], [11], [12], message-passing [13], [14], [15], [16] and relation-learning [17], [18], [19]. For detailed reviews on these categories and related methods, please refer to the recent survey [6].

Nevertheless, most existing HIN embedding approaches have primarily focused on designing encoding modules either supervised by node labels or committed to fitting node proximities [6], which naturally leads to satisfactory results on node classification and clustering tasks. Although some methods can acquire representations discriminative of node types and estimate the

Received 28 November 2022; revised 25 November 2023; accepted 26 January 2025. Date of publication 28 January 2025; date of current version 3 April 2025. This work was supported in part by the National Natural Science Foundation of China under Grant 62372146 and in part by the Zhejiang Provincial Natural Science Foundation of China under Grant LDT23F01015F01. Recommended for acceptance by I. Tsang. (*Corresponding author: Pengfei Jiao.*)

Xuan Guo, Danyang Shi, and Jie Li are with the College of Intelligence and Computing, Tianjin University, Tianjin 300350, China (e-mail: guoxuan@tju.edu.cn; shidanyang@tju.edu.cn; jieleo@tju.edu.cn).

Pengfei Jiao and Jinpeng Wang are with the School of Cyberspace, Hangzhou Dianzi University, Hangzhou 310018, China (e-mail: pjiao@hdu.edu.cn; jpwang@hdu.edu.cn).

Digital Object Identifier 10.1109/TPAMI.2025.3535763

connection likelihood between two nodes, they still ignore the higher-order event structures that encapsulate the generative semantics. As a result, they fall short in link prediction on HINs, as the absence of event information hinders the ability to learn underlying common properties among nodes. For example, as shown in Fig. 1, if two publications share some common terms, it can be inferred that these terms are highly related, and the associated authors likely have similar research interests. To learn these underlying properties, methods that define proximities via prior knowledge [9], [10], [20] such as meta-paths, need to combine a set of different predefined link patterns, limiting their generalization ability.

Inspired by the growing interest in Contrastive Learning within natural language processing [21], [22] and computer vision [23], [24], considerable efforts have been invested in self-supervised learning in the network domain [25], [26], [27], [28], [29], [30], [31]. Graph contrastive learning methods derive informative representations by maximizing the consistency between multiple augmented views of the original network [28], [29], [30], or by optimizing the contrastive loss on constructed positive and negative sample pairs/groups [25], [26], [27], [31]. Consequently, they operate without the need for labels and preserve more useful information than those proximity-preserving methods. In the context of HINs, a few attempts to leverage the contrastive learning manner have been made [25], [31], [32], [33], primarily focusing on view augmentation. However, they directly inherit the encoding modules of the classic embedding algorithms relying on predefined meta-paths, and can typically only generate embeddings for a specific type of nodes. This nature becomes the obstacle standing the path toward link prediction, as the links generally occur between nodes of different types in HINs. As mentioned above, they also overlook events, the natural and informative higher-order structures that can serve as contrastive samples at a suitable scale offering a wealth of useful underlying information for link prediction.

What's more, some network embedding methods are specifically designed for link prediction on HINs [16], [34], [35], [36], but most of them do not fully leverage the rich semantic information in HINs, especially the event structures. According to our above discussion, achieving better performance on link prediction in HINs requires modeling the generative process to simultaneously preserve both topological structures and higher-order semantic information. To the best of our knowledge, HeteHG-VAE [36] is the most qualified method complying with this criterion for link prediction. It models the events in HINs as hypergraphs with the variational autoencoder [37]. However, it trains embeddings by reconstructing the links between each central node and its associated nodes, capturing only the intra-event links and disregarding the inter-event relations that can sketch the generative process.

In a nutshell, **sketching the generative process via events benefits link prediction on heterogeneous information networks** by preserving the backbone of the entire network, especially in scenarios where large portions of links are missing. However, almost no existing algorithm fully realizes this idea. In this paper, we try to implement and verify this idea, introducing a new method named Contrastive Learning via

Events in Heterogeneous Information Networks (CLEH) for link prediction. Overall, CLEH can outline the generative process of HINs from individual nodes to the entire network through its embedding generation and higher-order contrastive learning components. During encoding, it constructs type-specific node-event bipartite graphs, effectively preserving relations between nodes and the events they participate in through its encoding and event pooling stages. We advocate leveraging a flow-based model as the encoder, given its ability to generate a flexible probability distribution in latent spaces with limited computational cost, enhancing the robustness and expressiveness of generated embeddings. To further capture the inter-event relations through different types of associated nodes, we design a higher-order contrastive learning mechanism at the event level. CLEH only needs two simple hyperparameters: the embedding dimension and the temperature parameter in the contrastive learning loss. We compare our CLEH with a number of embedding methods for link prediction on real-world networks. Both quantitative and qualitative experiments demonstrate the superiority of CLEH.

To summarize, our main contributions are as follows:

- We argue sketching the generative process could enhance HIN link prediction on HINs. For validation, we propose a HIN embedding method CLEH where the generative process is succinctly emulated by its embedding generation and higher-order contrastive learning modules.
- To fully preserve the higher-order event information, we design an event-level contrastive learning mechanism, which computes the loss based on sampled event pairs.
- The proposed CLEH is both effective and efficient, due to its concise yet natural model architecture. It has flexible practicability since it does not require additional balance parameters or prior information like meta-paths, except for two simple hyperparameters: the embedding size and the temperature parameter in the loss function.
- We conduct sufficient experiments on multiple real-world HIN datasets. The results on different link prediction tasks show the significant performance of CLEH compared to numerous SOTA baseline methods.

II. RELATED WORK

A. Classic HIN Representation Learning

Over the past decade, a plethora of methods have emerged for HIN representation learning [6]. Some methods preserve node proximities via techniques like random walks or direct reconstruction of low-order proximities. Metapath2vec [9] uses meta-paths with random walks to preserve node similarities and learn embeddings. JUST [11] extends the random walks with jumping/staying choices. PTE [38] transforms a HIN into multiple bipartite graphs and generates embeddings via matrix factorization. While extending these works, similar methods such as HIN2vec [10], HEER [12], SHINE [39], and others have been proposed. Recently, message-passing methods motivated by the development of Graph Neural Networks (GNNs) have been presented. R-GCN [16] addresses edge heterogeneity by learning multiple convolution layers. CompGCN [40] extends R-GCN by leveraging various entity-relation composition

operations. HAN [20] designs two levels of attention mechanisms to aggregate the information from different meta-path-based neighbors. Additional message-passing HIN embedding algorithms include HetSANN [13], HGT [14] and HGK-GNN [15]. Moreover, knowledge graphs are considered a special type of HINs [41]. Methods for knowledge graphs usually focus on designing triplet-based scoring functions, such as TransE [17], ConvE [18] and RHINE [19]. These methods are typically learned with labels or by reconstructing node proximities, yielding excellent results in node clustering and classification tasks. However, they show mediocre performance on link prediction, as the key higher-order structures in the HIN generative process, i.e., events, are ignored.

B. Embedding for Link Prediction

Embedding-based link prediction typically uses binary operators to combine learned endpoint embeddings into link embeddings. Subsequently, a model is trained on these link embeddings for making predictions [42], [43]. Most methods in this domain are trained to predict the scores of potential links based on observed network data. While many embedding methods, such as node2vec [42] and VGAE [44], have been designed for link prediction on homogeneous networks, methods tailored for HIN link prediction have become a focus of recent attention. For instance, MRIP [45] captures the correlation between different types of links and incorporates temporal features to enhance link prediction. RFG [34] combines general social patterns with network structure information. PME [35] unifies first-order and second-order proximities, addressing relation distribution imbalance via a loss-aware adaptive sampling approach. R-GCN [16] is developed specifically to handle the multi-relational characteristic of realistic knowledge bases. SLiCE [46] is a self-supervised model that conducts a contextual node prediction task for pre-training and link prediction for fine-tuning. These methods do not consider the higher-order event structures. To our best knowledge, HeteHG-VAE [36] is the first link prediction method that maps events to hypergraphs to capture both the higher-order structures and complex relations among nodes. However, it only reconstructs intra-event interactions and ignores inter-event interactions. As argued earlier, inter-event interactions are a good sketch of the generative process of HINs and contain useful information that can enhance link prediction.

C. Network Contrastive Learning

Contrastive learning has become a prominent unsupervised method for network embedding. Some network contrastive learning algorithms train representations by preserving consistency between different views of original networks [30]. The view augmentation processes in most methods are manually designed. For example, GraphCL [28] designs a set of augmentation ways on nodes, edges, attributes, and subgraphs. MVGRL [47] employs graph diffusion methods to generate augmented views. Some methods, like AD-GCL [29], attempt to generate augmented views automatically using a learnable model. More generally, network contrastive learning algorithms capture consistency by optimizing the losses relying on positive

TABLE I
MAINLY USED NOTATIONS

Notation	Description
$\mathcal{G} = \{V, E, \mathcal{T}, \phi\}$	The input HIN
V_t	The set of nodes of type t
$c_{\mathcal{G}}$	The central node type
$\varepsilon_v = \{V_v^{\varepsilon}, E_v^{\varepsilon}\}$	The event whose central node is v
$\mathcal{E}$	The set of events
$\mathcal{P}_{v,t}^+, \mathcal{N}_{v,t}^-$	The positive/negative sample set on type t for ε_v
$\mathbf{e}_v^t$	The type-specific embedding of ε_v on type t
$\mathcal{B}_t = \{V_t', E_t'\}$	The type-specific bipartite graph on type t
$\mathbf{z}_{v,t}, \bar{\mathbf{z}}_{v,t}$	The embedding of node v having type t
d	The dimension of learned embeddings
τ	The temperature parameter of contrastive loss
$\mathbf{W}_{\square}, \mathbf{b}_{\square}$	The learnable parameters
$f(\cdot), g(\cdot)$	The learnable transformation functions

and negative sample pairs. DGI [27] extends mutual information maximization [48] into the network domain by building local-global contrasting, using node-graph pairs as samples. GCC [26] takes subgraphs as samples and conducts a subgraph instance discrimination task for pre-training. In the context of HINs, HDGI [25] is the first to apply contrastive learning by combining DGI and HAN [20]. HeCo [31] employs the cross-view contrastive mechanism to extract the positive and negative embeddings from two views. SLiCE [46] and CPT-HG [33] leverage relation prediction as self-supervised pre-training tasks. However, none of these methods sketch the generative process or use events as contrastive samples, despite the natural suitability of the event structure for link prediction-oriented contrastive learning.

III. PRELIMINARIES

Here we give the formal definition of key notions. The main notations used in this paper are summarized in Table I.

Definition 1 (Heterogeneous Information Network): A heterogeneous information network (HIN) is denoted by a network $\mathcal{G} = \{V, E, \mathcal{T}, \mathcal{R}, \phi, \psi\}$ having various types of nodes and edges, where $V, E, \mathcal{T}, \mathcal{R}$ are the sets of nodes, edges, node types, and edge types, respectively. Each node $v \in V$ is mapped to a node type by ϕ , i.e., $\phi(v) \in \mathcal{T}$, and each edge $e \in E$ is mapped to an edge type by ψ , i.e., $\psi(e) \in \mathcal{R}$. Both ϕ and ψ are surjections.

Usually, the edge type is defined by the node type of the corresponding endpoints, so we will ignore $\mathcal{R}$ and ψ in the latter paper. We use $V_t = \{v | \phi(v) = t \wedge v \in V\}$ to denote the set of nodes having type t .

Definition 2 (Schema Graph): Given a HIN $\mathcal{G} = \{V, E, \mathcal{T}, \phi\}$, its schema graph $\mathcal{S}_{\mathcal{G}}$ summarizes the node and edge types. Formally, $\mathcal{S}_{\mathcal{G}} = \{V^{\phi}, E^{\phi}\}$ with $V^{\phi} = \{\phi(v) | v \in V\}$ and $E^{\phi} = \{(\phi(u), \phi(v)) | (u, v) \in E\}$.

Usually, the $\mathcal{S}_{\mathcal{G}}$ of a real-world HIN $\mathcal{G}$ is star-shaped, indicating the presence of a node type $t \in V^{\phi}$ to which all other node types are connected (Fig. 2(a)). We refer to this node type as the **central node type** $c_{\mathcal{G}}$. The connected node types to $c_{\mathcal{G}}$ are termed **associated node types**. In a star-schema HIN, each event is uniquely identified by a node of type $c_{\mathcal{G}}$. For instance, in the academic network, each publication is uniquely

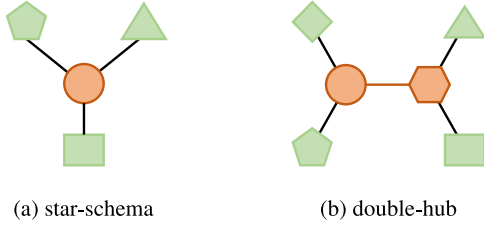


Fig. 2. Different schema graphs. (a) Usually, a real-world HIN has a star schema where events are uniquely identified by central nodes. We take star-schema HINs as the primer to introduce our method. (b) For a double-hub schema HIN, we use the hub node pairs to uniquely identify the events.

identified by a paper. Although for convenience, we assume that the input HIN pattern is star-shaped due to its universality, our method is not limited to such schemas. In the case of a non-star schema, alternative methods can be defined to identify events. For instance, in a HIN having a double-hub schema (Fig. 2(b)), we might use hub node pairs as event identifiers, and our method would still be applicable.

Definition 3 (Event): Given a HIN $\mathcal{G}$, for a node $v \in V$ satisfying $\phi(v) = c_{\mathcal{G}}$, its corresponding event is the subgraph $\varepsilon_v = \{V_v^{\varepsilon}, E_v^{\varepsilon}\}$ induced by v and all its neighbors N_v . Formally, $V_v^{\varepsilon} = \{v\} \cup N_v$ and $E_v^{\varepsilon} = \{(v, u) | u \in N_v\}$.

Each event is an example showing the semantics of $\mathcal{S}_{\mathcal{G}}$. In the event ε_v , we call v satisfying $\phi(v) = c_{\mathcal{G}}$ the **central node** and its neighbors **associated nodes**, respectively. Two events are connected if they have common associated nodes.

Definition 4 (HIN Embedding): Given a HIN $\mathcal{G}$, HIN embedding aims to map nodes V to d -dimensional representations $\{\mathbf{z}_v \in \mathbb{R}^d\}_{v \in V}$ with d much smaller than $|V|$. The learned representations should preserve critical information supporting downstream tasks (e.g., link prediction in this paper).

In this paper, we focus on learning HIN embeddings that facilitate link prediction. Specifically, on a HIN where partial edges are broken, we use the generated node embeddings to judge whether an edge exists between a pair of nodes in the original intact HIN.

Remark: This work encompasses terms featuring “**higher-order**”, and to avoid confusion, we offer clarifications for these terms. Essentially, “higher-order” is juxtaposed with “pair-wise”. “**Higher-order interaction/relation**” denotes relations in a network that simultaneously involve multiple nodes and their pairwise connections [49]. The fundamental unit in the network where higher-order interactions take place is termed an “**event**”, exemplified by paper publication events depicted in Fig. 1, constituting the academic network. The relations shared among nodes within an event are referred to as “**intra-event relations**”, while the connections between events through associated nodes are termed “**inter-event relations**”. Together, they outline the generative process of the network and encapsulate rich **higher-order semantics**. The former represents significant shared preferences among nodes, and the latter also conveys the similarity between events. To learn these semantics, we later introduce “**higher-order contrastive learning**”, which operates at the level of events. Its design ensures that the learning process occurs on node groups rather than node pairs.

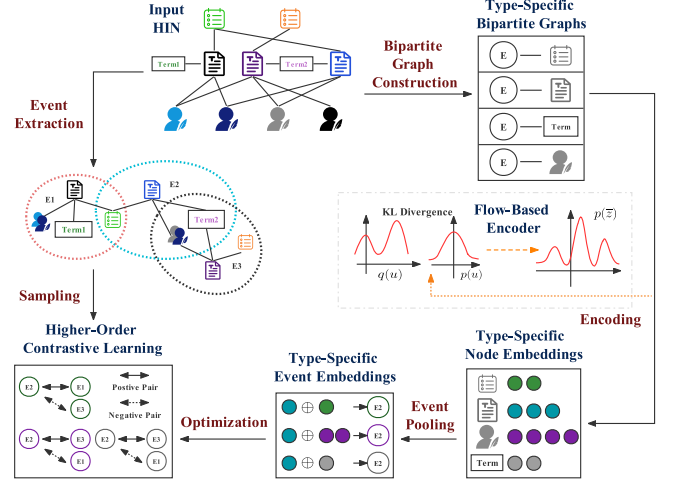


Fig. 3. The overview of CLEH. It mainly consists of two steps: the embedding generation (right) that leverages the normalizing flow mechanism to obtain embeddings, and the higher-order contrastive learning (left) on events. The two steps together sketch the generative process of the HIN.

IV. METHOD

In this section, we propose the HIN embedding model CLEH for link prediction. As shown in Fig. 3, CLEH mainly consists of two steps: embedding generation and higher-order contrastive learning. This two-step design is aligned with the generative process of HINs. From a top perspective, the higher-order contrastive learning step preserves the higher-order relations between events, which are the building blocks of HINs. From the bottom perspective, the embedding generation step models the intra-event relations among each event itself and its involved nodes. Correspondingly, CLEH learns two levels of representations: event embeddings and node embeddings. In the following, we present the details of the two steps in a top-down order.

A. Higher-Order Contrastive Learning

1) **Event Extraction:** Given a HIN $\mathcal{G} = \{V, E, \mathcal{T}, \phi\}$, we begin by extracting all of its higher-order semantic units, i.e., events [50]. According to Definition 3, each event in the star-schema HIN is a subgraph with a central node and multiple associated nodes. Thus, we build the event set as follows:

$$\mathcal{E} = \{\varepsilon_v | v \in V \wedge \phi(v) = c_{\mathcal{G}}\}. \quad (1)$$

Here each event is identified by the central node. For other HIN schemas, we could define identifiers instead of central nodes and corresponding associated nodes. See the example shown in Fig. 2(b).

2) **Contrastive Learning on Events:** Events are the building blocks of a HIN. These blocks are cemented together by some common associated nodes between event pairs. To model this characteristic of given HIN, the delicate event embeddings should be endowed with the ability to unveil connections between pairs of events and identify the associated node type linking two specific events.

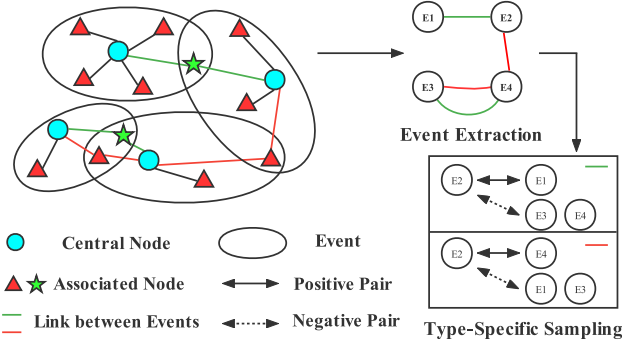


Fig. 4. Type-specific event sampling strategy for our higher-order contrastive learning. For a given associated node type, if an event sample is linked to the target event through at least one node of this type, we classify the sample as positive; otherwise, we consider it as negative.

To capture the relations between the events, we train the embeddings in a contrastive learning manner for both effectiveness and efficiency. As shown in Fig. 4, we construct positive and noise/negative samples for each associated node type $t \in \mathcal{T} - \{c_G\}$. Specifically, if event ε_u is connected to event ε_v via nodes of the associated type t , we regard the ε_u as a positive sample for ε_v on type t . Formally, the positive sample set for event ε_v on type t is defined as:

$$\mathcal{P}_v^t = \{\varepsilon_u | \varepsilon_u \in \mathcal{E} - \{\varepsilon_v\} \wedge V_t \cap V_v^\varepsilon \cap V_u^\varepsilon \neq \emptyset \wedge t \neq c_G\}. \quad (2)$$

Then we can capture the type-specific inter-event relations on associated node type t by optimizing the following contrastive loss function on $\mathcal{P}_v^t$:

$$\mathcal{L}_{\text{con}}^t = \sum_{\varepsilon_v \in \mathcal{E}} \left(-\log \sum_{\varepsilon_u \in \mathcal{P}_v^t} \exp(\text{sim}(\mathbf{h}_v^t, \mathbf{h}_u^t) / \tau) + \log \sum_{\varepsilon_a \in \mathcal{E}} \exp(\text{sim}(\mathbf{h}_v^t, \mathbf{h}_a^t) / \tau) \right), \quad (3)$$

where τ is the temperature parameter, and $\text{sim}(\cdot, \cdot)$ denotes cosine similarity. $\mathbf{h}_v^t$ is transformed from the type-specific embedding ϵ_v^t of event ε_v (got via the embedding generation step presented in Section IV-B) on type t via a multi-layer perceptron (MLP):

$$\mathbf{h}_v^t = \mathbf{W}_m^{(2)} \sigma(\mathbf{W}_m^{(1)} \epsilon_v^t + \mathbf{b}_m^{(1)}) + \mathbf{b}_m^{(2)}, \varepsilon_v \in \mathcal{E}, \quad (4)$$

where $\mathbf{W}_m^{(2)}$, $\mathbf{W}_m^{(1)}$, $\mathbf{b}_m^{(1)}$, and $\mathbf{b}_m^{(2)}$ are the parameters of MLP. $\sigma(\cdot)$ is a non-linear activation function which we set as the ReLU function in the whole method. This MLP for transformation is shared on all the associated node types.

The contrastive loss in (3), built upon Noise-Contrastive Estimation [51], extends InfoNCE [52] to events. Its primary objective is to differentiate between actual data samples and noise samples. In practice, negative sampling is often employed to diminish noise samples for expedited computation. We provide a type-specific event sampling strategy as shown in Fig. 4. For each positive sample set $\mathcal{P}_v^t$ of event ε_v , a corresponding

negative sample set $\mathcal{N}_v^t \subset \mathcal{E} - \mathcal{P}_v^t$ can be randomly extracted before training. Then the (3) is approximated as:

$$\mathcal{L}_{\text{con}}^t = \sum_{\varepsilon_v \in \mathcal{E}} \left(-\log \sum_{\varepsilon_u \in \mathcal{P}_v^t} \exp(\text{sim}(\mathbf{h}_v^t, \mathbf{h}_u^t) / \tau) + \log \sum_{\varepsilon_n \in \mathcal{N}_v^t} \exp(\text{sim}(\mathbf{h}_v^t, \mathbf{h}_n^t) / \tau) \right). \quad (5)$$

Indeed, using a limited number of negative samples can theoretically result in a consistent loss function [53], in some cases, empirically lead to superior results compared to using the entire set [54]. In our context, despite the random nature of negative sampling, it tends to choose events located farther from the target event, contributing to its effectiveness. By minimizing the contrastive loss $\mathcal{L}_{\text{con}}^t$, CLEH can retain information on whether two events are connected by t -type associated nodes. Then CLEH can capture the higher-order relations between events by combining the contrastive losses across all the associated node types.

B. Embedding Generation

1) *Type-Specific Bipartite Graph Construction*: In order to model the association between an event and various types of nodes in it, we construct a type-specific bipartite graph for each involved node type. Formally, the bipartite graph on the type t is defined as $\mathcal{B}_t = \{V_t', E_t'\}$ where $V_t' = \mathcal{E} \cup V_t$ and $E_t' = \{(\varepsilon_v, u) | \varepsilon_v \in \mathcal{E} \wedge u \in V_v^\varepsilon \wedge \phi(u) = t\}$ with events treated as nodes.

2) *Flow-Based Encoder*: On each type-specific bipartite graph, we generate embeddings for involved nodes. We leverage K -layer MLPs as the encoder to obtain embeddings. The k -th MLP layer is defined as :

$$\mathbf{z}_{v,t}^{(k)} = \sigma(\mathbf{W}_t^{(k)} \mathbf{z}_{v,t}^{(k-1)} + \mathbf{b}_t^{(k)}), v \in V_t \wedge 1 \leq k \leq K, \quad (6)$$

where $\mathbf{W}_t^{(k)}$ and $\mathbf{b}_t^{(k)}$ are the parameters of the MLP applied on bipartite graph $\mathcal{B}_t$. The initial representation $\mathbf{z}_{v,t}^{(0)}$ is the adjacency vector $\mathbf{a}_{v,t} \in \{0, 1\}^{|\mathcal{E}|}$ denoting which events are connected to node v in $\mathcal{B}_t$.

The generative process of real-world HINs is extremely complex, as even nodes having the same type are involved in different events for elusive reasons. To effectively capture this complexity, we draw inspiration from NF-VGA [55] and integrate the normalizing flow mechanism into our encoder for generating embeddings. Compared to other generative models like variational auto-encoder, the normalizing flow mechanism stacks multiple simple invertible processes and can reflect a complex multi-modal distribution based on the simple Gaussian distribution under similar computational cost. First, we apply actnorm and affine coupling [56] alternately on generated embeddings for multiple times. In each actnorm layer, the input d -dimension latent vector $\mathbf{z}$ is normalized as:

$$\mathbf{x} = \gamma \odot \mathbf{z} + \beta, \quad (7)$$

where γ and β are the normalization coefficients, and $\odot$ is the Harnard product. The affine coupling employed on the output of the actnorm layer is as follows:

$$\mathbf{y}_{1:l} = \mathbf{x}_{1:l}, \quad (8)$$

$$\mathbf{y}_{l+1:d} = \mathbf{x}_{l+1:d} \odot \exp(f(\mathbf{x}_{1:l})) + g(\mathbf{x}_{1:l}), \quad (9)$$

where $\mathbf{x}_{i:j}$ denotes the segment vector cut out from the i -th dimension to j -th dimension of $\mathbf{x}$. $f(\cdot)$ and $g(\cdot)$ are transformation functions mapping a l -dimension vector to a $(d-l)$ -dimension vector. The alternating procedures of (7)–(9) lead to a complex distribution being bijective with the Gaussian distribution.

Then we stack reverse procedures that are inverses of the above alternating procedures. The reverse procedures generate $\bar{\mathbf{z}}$ of which the distribution $p(\bar{\mathbf{z}})$ is the prior distribution of our generative model. Ideally, the real posterior distribution of the latent vector $q(\mathbf{z}|\mathbf{a})$ should be as close to the prior $p(\bar{\mathbf{z}})$ as possible. Therefore, we minimize the KL divergence between them. On embeddings $\mathbf{z}_{v,t} = \mathbf{z}_{v,t}^{(K)}$ of each type-specific bipartite graph, we implement the whole normalizing flow mechanism. The KL divergence loss on t -type bipartite graph to be minimized is:

$$\begin{aligned} \mathcal{L}_{\text{KL}}^t &= \sum_{v \in V_t} \text{KL}[q(\mathbf{z}_{v,t}|\mathbf{a}_{v,t})||p(\bar{\mathbf{z}}_{v,t})] \\ &= \sum_{v \in V_t} \left| \det \frac{\partial \mathbf{u}_{v,t}}{\partial \mathbf{z}_{v,t}} \right| \text{KL}[q(\mathbf{u}_{v,t})||p(\mathbf{u}_{v,t})], \end{aligned} \quad (10)$$

where $p(\mathbf{u}_{v,t})$ is a Gaussian distribution of the intermediate vector $\mathbf{u}_{v,t}$ and $q(\mathbf{u}_{v,t})$ is the auxiliary distribution that should be close to $p(\mathbf{u}_{v,t})$. $|\det \frac{\partial \mathbf{u}_{v,t}}{\partial \mathbf{z}_{v,t}}|$ is the Jacobian determinant.

3) *Event Pooling*: To facilitate learning on events, aggregating node information within events into event embeddings is essential. The central node serves as the core of an event, with associated nodes influencing its characteristics. Thus, we design the following pooling method to get type-specific event embeddings that are used in (4):

$$\epsilon_v^t = \bar{\mathbf{z}}_{v,c_g} \oplus \sum_{u \in V_t \cap N_v} \bar{\mathbf{z}}_{u,t}, \epsilon_v \in \mathcal{E} \wedge t \neq c_g, \quad (11)$$

where $\oplus$ is the concatenation operation. To anchor its neighbors in the learning process, the complete information of the event identifier (i.e., the central node) is retained. Sum-pooling is employed on associated nodes for its ability to preserve the most neighborhood information, serving as a parameter-free graph pooling method [57]. For non-star schemas, corresponding methods can be defined to represent the identifiers. Taking the double-hub schema shown in Fig. 2(b) as an example, the embeddings of the two hub nodes for each event could be summed up or concatenated as its identification representation.

C. Joint Optimization

CLEH needs to finely estimate the inter-event and intra-event relations with the flow mechanism modeling the complex distribution in the latent space. Therefore, we optimize CLEH by minimizing the contrastive learning losses and the KL

Algorithm 1: The Procedure of CLEH.

Input: the HIN $\mathcal{G}$, temperature parameter τ and the embedding size d .

Output: Node embedding: $\{\mathbf{z}_{v,t}\}_{v \in V}$.

- 1: Process the HIN $\mathcal{G}$ to $\mathcal{B}_t$;
 - 2: Build the event set $\mathcal{E}$;
 - 3: Construct $\mathcal{P}_v^t$ and $\mathcal{N}_v^t$ for each event;
 - 4: **for** $epoch = 1$ to T **do**
 - 5: Project $\mathbf{a}_{v,t}$ to $\mathbf{z}_{v,t}^{(k)}$ with MLP;
 - 6: Feed the $\mathbf{z}_{v,t}^{(k)}$ to normalizing flow via (7)–(9) and their reverse procedures;
 - 7: Calculate KL divergence through (10);
 - 8: Pooling the type-specific events embedding ϵ_v^t via (11);
 - 9: Project ϵ_v^t to $\mathbf{h}_v^t$ with MLP;
 - 10: Calculate contrastive loss via (5);
 - 11: Update the CLEH via (12);
 - 12: **end for**
 - 13: **return** $\{\mathbf{z}_{v,t}\}_{v \in V}$;
-

divergence losses jointly:

$$\mathcal{L} = \sum_{t \in \mathcal{T} - \{c_g\}} \mathcal{L}_{\text{con}}^t + \sum_{t \in \mathcal{T}} \mathcal{L}_{\text{KL}}^t. \quad (12)$$

In summary, CLEH concisely depicts the bone of the input HIN via its embedding generation and higher-order contrastive learning modules. CLEH only relies on two hyperparameters: the embedding dimension and temperature parameter of the contrastive loss. For easier understanding, the pseudo-code of CLEH is shown in Algorithm 1.

D. Time Complexity Analysis

Constructing type-specific bipartite graphs costs time $O(|E|)$ for traversing all edges once. CLEH generates d -dimensional embeddings for nodes and events, with the number of nodes typically outnumbering events. Thus, the time complexity of embedding generation via the flow-based encoder is $O(|V|d)$. Event sampling for all events takes time $O(|\mathcal{E}|)$. Computing the contrastive learning loss averages $O(D_{\text{avg}}|\mathcal{E}|d)$, where D_{avg} is the average degree of central nodes. Computing the KL divergence loss costs $O(|V|d)$. Note that $O(|\mathcal{E}|) < O(|V|) < O(D_{\text{avg}}|\mathcal{E}|) = O(|E|)$. Thus, the total time complexity of CLEH is linear with $|E|$, which ensures its scalability.

V. EXPERIMENTS

In this section, we conduct both quantitative and qualitative experiments on HIN link prediction tasks for evaluation. A number of SOTA baselines are implemented for comparison with our CLEH on several real-world datasets.

A. Datasets

We conduct experiments on three real-world datasets from various domains, which are subsets of the open benchmarks in [6]. The statistics of them are shown in Table II.

TABLE II
STATISTICS OF THE DATASETS

Dataset	# Nodes				# Edges		
DBLP	Author: 4,057	Paper: 14,328	Venue: 20	Term: 7,723	P-A (1:n): 19,645	P-V (1:1): 14,328	P-T (1:n): 85,810
PubMed	Chemical: 26,52	Gene: 13,561	Species: 2,863	Disease: 20,163	G-C (1:n): 31,277	G-D (1:n): 25,962	G-S (1:1): 3,155
Yelp	Location: 39	Business: 7,474	Star: 9	Phrase: 74,943	B-L (1:1): 7,474	B-P (1:n): 3,981,469	B-S (1:1): 7,474

- *DBLP*, an academic network in computer science, comprises 4,057 authors (A), 14,328 papers (P), 20 venues (V), and 7,723 terms (T). Papers serve as the central node type while the others are associated node types. In total, there are 119,783 edges among these different types of nodes.
- *PubMed*, a phrase co-occurrence network extracted from biomedical publications, encompassing 26,522 chemicals (C), 13,561 genes (G), 2,863 species (S), and 20,163 diseases (D). Genes are considered central nodes, and the other three types of nodes are regarded as associated nodes. 60,394 edges exist in this network.
- *Yelp* is a business review network with 39 locations (L), 7,474 businesses (B), 9 star rates (S), and 74,943 review phrases (P). Business nodes are the central nodes and the other types of nodes are associated nodes. This network has 3,996,417 edges in total, and the vast majority of them occur between businesses and phrases.

B. Baselines

We compare our method with a number of popular and state-of-the-art unsupervised representation learning methods. The implemented popular homogeneous network embedding methods on link prediction are as follows:

- *DeepWalk* [58] leverages random walks to capture proximities and applies a language model for embedding generation. Its time complexity is bounded by $O(|V|)$.
- *Node2vec* [42] extends DeepWalk by introducing hyperparameters that bias random walks, balancing breadth-first and depth-first search. Time complexity is $O(|V|)$.
- *LINE* [59] trains representations via reconstructing links and the overlapping degree of two-hop adjacency neighborhoods on sampled node pairs. Its time cost is $O(|E|)$.
- *HSRL* [60] compresses input network into smaller networks with decreasing sizes. Embeddings are generated and aggregated on compressed networks via an arbitrary classic method. It takes longer than the selected classic approaches but can be parallelized on compressed graphs.

The employed HIN embedding methods are as follows:

- *TransE* [17], designed for knowledge graphs, translates relationships into triplet score computation based on node embeddings. It takes a time complexity of $O(|N|)$.
- *PTE* [38] decomposes the HIN into multiple bipartite networks based on edge types and learns embeddings by reconstructing the first-order proximity on all bipartite networks. It exhibits a time complexity of $O(|E|)$.
- *Metapath2vec* [9] extends DeepWalk to HINs by employing meta-path-based random walks and uses a language model to generate embeddings. Its time complexity is bounded by $O(|V|)$.

- *HIN2vec* [10] learns embeddings by predicting whether there is a specific meta-path between two nodes using homogeneous random walks to obtain positive samples. It exhibits a time complexity of $O(|V|)$.
- *ConvE* [18] uses deep neural models based on the 2D convolution over embeddings to score the triplets in knowledge graphs instead of distance/similarity functions. It takes a time complexity of $O(|N|)$.
- *JUST* [11] biases random walks with jump and stay strategies to alleviate the reliance on predefined meta-paths and balance the node distribution w.r.t node types. The time complexity of JUST is $O(|V|)$.
- *R-GCN* [16] considers edge heterogeneity by learning multiple graph convolution matrices on each edge type. Its GCN [61] architecture has complexity $O(|E|)$.
- *AspEm* [62] defines the aspect in HIN as a subgraph of the network schema and trains embeddings via reconstructing the aspect-based proximity. Its time complexity amounts to $O(|E|)$.
- *HGT* [14] leverages the Transformer-like self-attention architecture to learn embeddings by reconstructing the triplet of two node types and the edge type. HGT incurs a time complexity of $O(|V|)$.
- *CPT-HG* [33] designs both link-level and subgraph-level pre-training tasks, and proposes several negative sampling methods on both tasks for contrastive learning. Its GNN architecture has complexity $O(|E|)$.
- *HeteHG-VAE* [36] uses a variational auto-encoder and attention mechanism based on events to represent heterogeneous information networks in an unsupervised way. The computational cost of HeteHG-VAE is $O(|E|)$.

Here we provide the time complexities of these methods w.r.t. the statistics of the input network for reference. However, it's important to note that judging their efficiency solely based on these complexities might be challenging. The methods vary significantly in terms of hyperparameters, learning paradigms, and training approaches, making it difficult to establish a unified standard for comparison. Additionally, it's highlighted that none of the baselines sketch the generative process of HINs. HeteHG-VAE captures intra-event interactions but does not fully learn those inter-event relations.

C. Model and Environmental Settings

On all the datasets, we train CLEH for 50 epochs via Adam optimizer [63]. The learning rate is set to 0.01. For the flow-based encoder, we use 1-layer MLP and repeat the alternating procedures, i.e., (7)–(9) and their reverse procedures for 4 times. τ is set to 0.9 on DBLP and 0.5 on the other two datasets. The embedding size of all algorithms is set to 128 by default. Besides, our

TABLE III
LINK PREDICTION RESULTS ON DBLP NETWORK WITH DIFFERENT RATIOS OF REMOVED LINKS

Removal ratio	20%		40%		60%		80%		Mean $\pm$ Std.	
Metric	AUC	AP	AUC	AP	AUC	AP	AUC	AP	AUC	AP
DeepWalk	0.6875	0.6555	0.6722	0.6293	0.6306	0.5967	0.5471	0.5490	0.6344 ± 0.0629	0.6076 ± 0.0459
node2vec	0.7197	0.7152	0.7010	0.6877	0.6524	0.6461	0.6302	0.6136	0.6758 ± 0.0416	0.6657 ± 0.0448
LINE	0.7682	0.7667	0.7610	0.7553	0.7444	0.7325	0.6915	0.6819	0.7413 ± 0.0346	0.7341 ± 0.0376
HSRL	0.7099	0.7052	0.7163	0.7030	0.6857	0.6663	0.5969	0.6049	0.6772 ± 0.0551	0.6699 ± 0.0468
TransE	0.7667	0.7685	0.7137	0.7128	0.6193	0.6128	0.5119	0.5084	0.6529 ± 0.1120	0.6506 ± 0.1146
PTE	0.5577	0.5549	0.5556	0.5557	0.5478	0.5426	0.5421	0.5415	0.5508 ± 0.0072	0.5487 ± 0.0077
metapath2vec	0.7496	0.7042	0.6761	0.6362	0.6151	0.5889	0.5592	0.5552	0.6500 ± 0.0818	0.6211 ± 0.0646
HIN2vec	0.7249	0.7122	0.6660	0.6522	0.7353	0.7328	0.5505	0.5262	0.6692 ± 0.0848	0.6559 ± 0.0929
ConvE	0.7279	0.7503	0.7211	0.7408	0.6301	0.6641	0.6460	0.6618	0.6813 ± 0.0504	0.7043 ± 0.0479
JUST	0.4988	0.5012	0.5032	0.5133	0.5019	0.5196	0.4982	0.5202	0.5005 ± 0.0024	0.5136 ± 0.0088
R-GCN	0.5011	0.4961	0.5024	0.5002	0.5026	0.5011	0.5006	0.4984	0.5017 ± 0.0010	0.4990 ± 0.0022
AspEm	0.5310	0.5969	0.4510	0.5073	0.3728	0.4484	0.3603	0.4592	0.4288 ± 0.0791	0.5030 ± 0.0677
HGT	0.6187	0.6693	0.5552	0.6120	0.4977	0.5582	0.4808	0.5247	0.5381 ± 0.0625	0.5911 ± 0.0634
CPT-HG	0.8627	0.8496	0.8124	0.8027	0.7844	0.7863	0.7452	0.7320	0.8012 ± 0.0494	0.7927 ± 0.0485
HeteHG-VAE	0.8445	<u>0.8592</u>	<u>0.8424</u>	<u>0.8591</u>	<u>0.8370</u>	<u>0.8576</u>	<u>0.8196</u>	<u>0.8465</u>	0.8359 ± 0.0113	0.8556 ± 0.0061
CLEH	<u>0.8587</u>	0.8712	0.8514	0.8661	0.8473	0.8599	0.8420	0.8592	0.8499 ± 0.0070	0.8641 ± 0.0057

TABLE IV
LINK PREDICTION RESULTS ON PUBMED NETWORK WITH DIFFERENT RATIOS OF REMOVED LINKS

Removal ratio	20%		40%		60%		80%		Mean $\pm$ Std.	
Metric	AUC	AP	AUC	AP	AUC	AP	AUC	AP	AUC	AP
DeepWalk	0.7346	0.7019	0.7233	0.7031	0.6796	0.6766	0.6440	0.6483	0.6954 ± 0.0417	0.6825 ± 0.0259
node2vec	0.7580	0.7437	0.7423	0.7326	0.7213	0.7146	0.6826	0.6735	0.7261 ± 0.0326	0.7161 ± 0.0308
LINE	0.7771	0.8108	0.7687	0.7930	<u>0.7471</u>	0.7612	0.7037	0.7076	0.7492 ± 0.0328	0.7682 ± 0.0453
HSRL	0.7682	0.7739	0.7540	0.7586	0.7201	0.7287	0.6758	0.6806	0.7295 ± 0.0411	0.7355 ± 0.0411
TransE	0.7027	0.7609	0.6901	0.7698	0.6481	0.7551	0.5699	0.6953	0.6527 ± 0.0599	0.7453 ± 0.0339
PTE	0.5286	0.5823	0.5268	0.5852	0.5265	0.5908	0.5242	0.6010	0.5265 ± 0.0018	0.5898 ± 0.0082
metapath2vec	0.8054	0.7984	0.7768	0.7620	0.7384	0.7259	<u>0.7256</u>	0.7038	0.7616 ± 0.0364	0.7475 ± 0.0415
HIN2vec	0.6383	0.5512	0.4831	0.5649	0.4864	0.5575	0.6834	0.6536	0.5728 ± 0.1033	0.5818 ± 0.0482
ConvE	0.7483	0.7710	0.7590	0.7760	0.7401	0.7445	0.7002	0.6937	0.7369 ± 0.0257	0.7463 ± 0.0377
JUST	0.4973	0.5355	0.5020	0.5510	0.5053	0.5610	0.5010	0.5548	0.5014 ± 0.0033	0.5506 ± 0.0109
R-GCN	0.5012	0.4957	0.5040	0.4975	0.5004	0.4973	0.5088	0.5051	0.5036 ± 0.0038	0.4989 ± 0.0042
AspEm	0.3390	0.4830	0.2968	0.4461	0.3113	0.4709	0.3347	0.4939	0.3205 ± 0.0199	0.4735 ± 0.0205
HGT	0.7063	0.7877	0.6837	0.7619	0.6340	0.7264	0.5253	0.6367	0.6373 ± 0.0806	0.7282 ± 0.0660
CPT-HG	0.8586	0.8758	<u>0.8147</u>	0.8256	0.7427	0.7542	0.7070	0.7144	0.7808 ± 0.0686	0.7925 ± 0.0721
HeteHG-VAE	0.8452	0.8850	0.8024	<u>0.8425</u>	0.7458	<u>0.7847</u>	0.7023	<u>0.7539</u>	0.7739 ± 0.0627	0.8165 ± 0.0586
CLEH	<u>0.8483</u>	<u>0.8789</u>	0.8440	0.8568	0.8457	0.8605	0.8388	0.8519	0.8442 ± 0.0040	0.8620 ± 0.0118

model has no additional hyperparameters. For metapath2vec, we try all the binary combinations of second-order meta-paths (e.g., AVPVA) and choose the one combination leading to the best results. All parameters of baselines are fine-tuned via grid search.

All the experiments are conducted on a Ubuntu 18.04.5 LTS server equipped with an Intel(R) Xeon(R) Silver 4210 CPU @ 2.20 GHz, 512 GB RAM. The software used includes Pytorch version 1.8.1, scikit-learn version 0.24.1, and Python version 3.7. The implementation of CLEH is available at <https://anonymous.4open.science/r/CLEH-5B6B>.

D. Link Prediction

In this subsection, we demonstrate the effectiveness of the proposed method by presenting the results on both general and typed link prediction tasks.

1) *General Link Prediction*: To implement general link prediction experiments, we follow the experimental strategy recommended in previous literature [42], [64]. 20% – 80% of existing edges in a given HIN are randomly removed. All models are trained on the remaining HIN. The removed edges are treated as positive examples, and an equal number of non-edges are sampled as negative examples for testing. Edge features are obtained by computing the Hadamard operator of the corresponding endpoints. A binary logistic regression classifier is trained on these edge features for link prediction. Quantitative evaluation involves the application of area under ROC curve (AUC) and average precision (AP). The whole process is repeated 10 times and we report the average results.

The results of all compared methods on DBLP, PubMed and Yelp datasets are shown in Tables III, IV, and V, respectively. Mean and standard deviation are reported for all removal ratio settings to showcase the robustness of each method. The best

TABLE V
LINK PREDICTION RESULTS ON YELP NETWORK WITH DIFFERENT RATIOS OF REMOVED LINKS

Removal ratio	20%		40%		60%		80%		Mean $\pm$ Std.	
Metric	AUC	AP	AUC	AP	AUC	AP	AUC	AP	AUC	AP
DeepWalk	0.5271	0.4850	0.5329	0.4877	0.5294	0.4843	0.5165	0.4790	0.5265 $\pm$ 0.0071	0.4840 $\pm$ 0.0036
node2vec	0.5263	0.4809	0.5379	0.4893	0.5438	0.4963	0.5302	0.5302	0.5346 $\pm$ 0.0078	0.4992 $\pm$ 0.0216
LINE	0.8059	0.8067	0.7843	0.7832	0.7632	0.7624	0.7312	0.7310	0.7712 $\pm$ 0.0318	0.7708 $\pm$ 0.0321
HSRL	0.5921	0.5358	0.5901	0.5326	0.5713	0.5175	0.5516	0.5015	0.5763 $\pm$ 0.0189	0.5219 $\pm$ 0.0157
TransE	0.7351	0.6975	0.7410	0.7035	0.7416	0.6987	0.7667	0.7347	0.7461 $\pm$ 0.0140	0.7086 $\pm$ 0.0176
PTE	0.5171	0.5262	0.5149	0.5277	0.5164	0.5256	0.5157	0.5285	0.5160 $\pm$ 0.0009	0.5270 $\pm$ 0.0013
metapath2vec	0.7984	0.8054	0.7620	0.7768	0.7259	0.7384	0.7038	0.7256	0.7475 $\pm$ 0.0415	0.7616 $\pm$ 0.0364
HIN2vec	0.8526	0.8645	0.8411	0.8558	0.8397	0.8552	0.8240	0.8448	0.8394 $\pm$ 0.0118	0.8551 $\pm$ 0.0081
ConvE	0.8544	0.8617	0.8331	0.8430	0.8059	0.7942	0.7730	0.7861	0.8166 $\pm$ 0.0352	0.8213 $\pm$ 0.0369
JUST	0.5000	0.5024	0.5001	0.5016	0.5003	0.5034	0.5001	0.5084	0.5001 $\pm$ 0.0001	0.5040 $\pm$ 0.0031
R-GCN	0.5063	0.5030	0.5077	0.5038	0.5063	0.5024	0.5038	0.5002	0.5060 $\pm$ 0.0016	0.5024 $\pm$ 0.0015
AspEm	0.3390	0.4830	0.7155	0.7096	0.6589	0.6361	0.5840	0.5559	0.5744 $\pm$ 0.1659	0.5962 $\pm$ 0.0981
HGT	0.8254	0.8403	0.8158	0.8315	0.7857	0.8024	0.7545	0.7624	0.7954 $\pm$ 0.0321	0.8092 $\pm$ 0.0351
CPT-HG	0.8522	0.8673	0.8021	0.8124	0.7682	0.7720	0.7148	0.7052	0.7843 $\pm$ 0.0578	0.7892 $\pm$ 0.0683
HeteHG-VAE	0.8379	0.8547	0.8376	0.8458	0.8324	0.8475	<u>0.8301</u>	<u>0.8450</u>	0.8345 $\pm$ 0.0039	0.8483 $\pm$ 0.0044
CLEH	0.8483	0.8617	0.8452	0.8574	0.8439	0.8588	0.8381	0.8552	0.8439 $\pm$ 0.0043	0.8583 $\pm$ 0.0027

results for each removal ratio and metric are bolded, and the second-best results are underlined.

Overall, our CLEH achieves the best results over almost all the removal ratios on each dataset. HeteHG-VAE is second only to CLEH in performance on DBLP and PubMed as both methods focus on preserving higher-order relations among nodes. The performance of CPT-HG is notable with a small link removal ratio, benefiting from the co-training of both link-level and subgraph-level prediction tasks. However, its performance declines significantly as more links are removed before training. The performance of HIN2vec, ConvE, and HGT on Yelp is quite competitive and significantly better than their performance on the DBLP and PubMed. This is attributed to the imbalanced distribution of different edge types. Since the business-phrases type occupies a substantial part, the designs of HIN2vec, ConvE, and HGT become customised as almost only one type of links need to be reconstructed.

Compared with other methods having competitive performance, CLEH gets much closer results when 20% of edges are removed to those when 80% of edges are removed. We attribute this to the event-based higher-order contrastive learning of CLEH. When a large number of links are removed in a HIN, the original topology within events is heavily broken, but the bone within events and most of the connections between events still remain. Preserving this critical remaining information is the foundation for an embedding method to facilitate link prediction in HINs. CLEH achieves this via the type-specific bipartite graphs and the contrastive learning based on them. HeteHG-VAE can only capture the bone within events because it directly reconstructs the hyperlinks between nodes. Thus, as a larger ratio of edges are removed, HeteHG-VAE gradually loses its effectiveness. In summary, the key factor that makes CLEH outperform HeteHG-VAE is its ability to sketch the generative process of HINs, specifically building relations between different types of nodes and events.

2) *Typed Link Prediction*: In real-world applications, predicting the links of a specific type is more popular than general

TABLE VI
TYPED LINK PREDICTION RESULTS ON DBLP NETWORK WITH 20% OF LINKS REMOVED

Edge Type	P-A		P-V		P-T	
Metric	AUC	AP	AUC	AP	AUC	AP
DeepWalk	0.5865	0.6074	0.5477	0.5475	0.6423	0.6450
node2vec	0.5241	0.5074	0.6789	0.6781	0.7078	0.7177
LINE	0.6772	0.6778	0.6475	0.6362	0.7875	0.7901
HSRL	0.6277	0.6253	0.6805	0.6877	0.7243	0.7324
TransE	0.6253	0.6423	0.6600	0.6584	0.7745	0.7856
PTE	0.4157	0.4220	0.4275	0.4123	0.5576	0.5574
metapath2vec	0.6412	0.6327	0.6123	0.6147	0.7477	0.7375
HIN2vec	0.6175	0.6275	0.5978	0.6075	0.7162	0.7257
ConvE	0.6023	0.5975	0.6274	0.6025	0.7872	0.7886
JUST	0.4276	0.4352	0.4057	0.4125	0.5325	0.5245
R-GCN	0.4689	0.4752	0.4073	0.3856	0.5447	0.5023
AspEm	0.5695	0.5478	0.5024	0.4896	0.5957	0.6087
HGT	0.6345	0.6247	0.5423	0.5314	0.6450	0.6812
CPT-HG	0.7280	0.7457	<u>0.7248</u>	<u>0.6824</u>	0.8900	<u>0.8951</u>
HeteHG-VAE	0.7056	0.7042	0.7250	0.6710	0.8712	0.8648
CLEH	<u>0.7116</u>	<u>0.7394</u>	0.7167	0.6713	<u>0.8815</u>	0.8966

link prediction. Therefore, we conduct typed link prediction experiments, randomly removing 20% of edges for each edge type in an input HIN. Then we apply the mentioned strategy for general link prediction to generate embeddings and make predictions. Still, we use AUC and AP as the evaluation metrics and report the average results of 10 repeated experiments.

To save space, we only report the results on DBLP, PubMed and Yelp networks with 20% typed links removed in Tables VI, VII, and VIII, respectively. The best results for each edge type and metric are bolded, and the second-best results are underlined.

Observing the results, CLEH consistently outperforms other methods, excluding CPT-HG on some edge types. According to the results on general link prediction, CPT-HG loses competitiveness when more links are removed. Thus, HeteHG-VAE is the only competitive method among the baselines on typed link prediction compared with CLEH. The other HIN embedding methods even perform worse than some methods

TABLE VII
TYPED LINK PREDICTION RESULTS ON PUBMED NETWORK WITH 20% OF
LINKS REMOVED

Edge Type	G-C		G-D		G-S	
	AUC	AP	AUC	AP	AUC	AP
DeepWalk	0.7652	0.7004	0.6975	0.7058	0.6857	0.7144
node2vec	0.7214	0.7052	0.7542	0.7870	0.7657	0.7490
LINE	0.7478	0.7954	0.7658	0.8047	0.8147	0.8324
HSRL	0.7483	0.7522	0.7321	0.7704	0.7657	0.7880
TransE	0.7087	0.7423	0.7425	0.7642	0.7425	0.7754
PTE	0.5744	0.5997	0.5634	0.5732	0.5786	0.6012
metapath2vec	0.7814	0.8023	0.7412	0.8146	0.7892	0.8174
HIN2vec	0.5280	0.5574	0.5463	0.5717	0.5057	0.5671
ConvE	0.7326	0.7841	0.7672	0.7974	0.7563	0.7880
JUST	0.5789	0.5277	0.4978	0.5274	0.5174	0.5454
R-GCN	0.4372	0.4777	0.4573	0.4971	0.4670	0.5042
AspEm	0.4321	0.4787	0.4587	0.4875	0.3524	0.4957
HGT	0.7527	0.7944	0.8017	0.7925	0.6985	0.7820
CPT-HG	0.8742	0.8923	0.8543	0.8994	0.8587	0.8720
HeteHG-VAE	0.8542	0.8756	<u>0.8652</u>	0.8750	<u>0.8524</u>	0.8870
CLEH	<u>0.8654</u>	<u>0.8890</u>	0.8667	<u>0.8922</u>	0.8468	<u>0.8787</u>

TABLE VIII
TYPED LINK PREDICTION RESULTS ON YELP NETWORK WITH 20% OF LINKS
REMOVED

Edge Type	B-L		B-P		B-S	
	AUC	AP	AUC	AP	AUC	AP
DeepWalk	0.7523	0.7768	0.7648	0.7584	0.6376	0.6327
node2vec	0.7621	0.7977	0.7843	0.7814	0.6822	0.6754
LINE	0.8224	0.8476	0.8072	<u>0.8574</u>	0.7350	0.7527
HSRL	0.7827	0.7975	0.7620	0.7746	0.6827	0.6978
TransE	0.8347	0.8475	0.8137	0.8217	0.6875	0.7024
PTE	0.6872	0.6575	0.6570	0.6423	0.5026	0.5234
metapath2vec	0.8757	0.8627	0.8376	0.8275	0.7384	0.7259
HIN2vec	0.5937	0.6072	0.6789	0.5572	0.5871	0.4823
ConvE	0.8573	0.8372	0.8173	0.8027	0.7317	0.7037
JUST	0.5573	0.5627	0.5375	0.5073	0.5077	0.4873
R-GCN	0.5137	0.5276	0.4757	0.4975	0.4527	0.4377
AspEm	0.5027	0.4976	0.4592	0.4679	0.4875	0.4357
HGT	0.7625	0.7924	0.7019	0.7246	0.7210	0.6857
CPT-HG	<u>0.9420</u>	<u>0.9278</u>	0.8584	0.8423	0.7920	<u>0.7396</u>
HeteHG-VAE	0.9325	0.9026	<u>0.8475</u>	0.8513	0.7650	0.7274
CLEH	0.9426	0.9311	0.8473	0.8612	<u>0.7889</u>	0.7370

designed for homogeneous networks. This observation implies that the higher-order event structures help typed link prediction.

On predicting P-V edges in the DBLP network and G-S edges in the PubMed network, we find that the results of HeteHG-VAE are slightly larger than those of CLEH. This is because there is at most only one edge having the P-V or G-S type in an event. For CLEH, these two types of edges are too sparse in events, making them more challenging to predict. In contrast, HeteHG-VAE specifically reconstructs edges of each type and performs better on those extremely sparse typed links. Similarly, HeteHG-VAE also performs well in predicting B-P edges in the Yelp network because B-P edges are extremely dense compared to the other two types of edges.

E. Event Relation Prediction

Here we aim to verify whether CLEH and the baselines effectively implement the core idea of this paper: sketching

TABLE IX
EVENT RELATION PREDICTION RESULTS ON DBLP NETWORK WITH 20% OF
INTER-EVENT RELATIONS REMOVED

Relation Type	E-A-E		E-V-E		E-T-E	
	AUC	AP	AUC	AP	AUC	AP
DeepWalk	0.5042	0.5103	0.4976	0.4823	0.5009	0.4910
node2vec	0.5172	0.5327	0.5878	0.5831	0.5326	0.5401
LINE	0.6800	0.6927	0.6107	0.6112	0.6072	0.6379
HSRL	0.6173	0.6427	0.6298	0.6372	0.6578	0.6670
TransE	0.6427	0.6557	0.6372	0.6727	0.6827	0.7036
PTE	0.4272	0.4573	0.4758	0.4852	0.5427	0.5384
metapath2vec	0.6728	0.6976	0.6572	0.6643	0.7024	0.7135
HIN2vec	0.5972	0.6075	0.5721	0.5768	0.6072	0.6176
ConvE	0.6275	0.6379	0.6422	0.6417	0.6927	0.6872
JUST	0.4427	0.4547	0.4373	0.4272	0.5042	0.5153
R-GCN	0.4756	0.4963	0.4527	0.4827	0.5243	0.5276
AspEm	0.5427	0.5757	0.5142	0.5001	0.5421	0.5623
HGT	0.7487	0.7544	0.6875	0.6971	0.7027	0.7247
CPT-HG	0.7743	0.7548	0.7826	0.7997	0.7410	0.7322
HeteHG-VAE	<u>0.7825</u>	<u>0.7921</u>	<u>0.8042</u>	<u>0.8254</u>	<u>0.8210</u>	<u>0.8371</u>
CLEH	0.8249	0.8415	0.8457	0.8628	0.8920	0.9073

TABLE X
EVENT RELATION PREDICTION RESULTS ON PUBMED NETWORK WITH 20% OF
INTER-EVENT RELATIONS REMOVED

Relation Type	E-C-E		E-D-E		E-S-E	
	AUC	AP	AUC	AP	AUC	AP
DeepWalk	0.5942	0.5873	0.5937	0.5881	0.5654	0.5820
node2vec	0.6824	0.6953	0.6626	0.6701	0.6450	0.6478
LINE	0.6785	0.6897	0.6537	0.6699	0.6473	0.6290
HSRL	0.6073	0.6177	0.5781	0.5923	0.6209	0.6357
TransE	0.6577	0.6628	0.6345	0.6449	0.6901	0.6875
PTE	0.4825	0.4920	0.5000	0.5179	0.4753	0.4992
metapath2vec	<u>0.7933</u>	<u>0.8127</u>	0.7407	0.7366	0.8022	0.8150
HIN2vec	0.6009	0.6123	0.5848	0.5938	0.6403	0.6570
ConvE	0.6824	0.6850	0.6377	0.6301	0.6637	0.6511
JUST	0.4574	0.4628	0.4414	0.4539	0.4932	0.5077
R-GCN	0.5748	0.5770	0.5490	0.5761	0.5332	0.5270
AspEm	0.5200	0.5324	0.4957	0.4803	0.5342	0.5570
HGT	0.7273	0.7480	0.6954	0.6783	0.7594	0.7623
CPT-HG	0.7042	0.7129	0.6954	0.7091	0.7435	0.7411
HeteHG-VAE	0.7825	0.7921	<u>0.8042</u>	<u>0.8254</u>	<u>0.8210</u>	<u>0.8371</u>
CLEH	0.8583	0.8620	0.8376	0.8479	0.8872	0.8970

the generative process of HINs. To this end, we conduct the following event relation prediction experiments. We randomly remove 20% of inter-event relations for each relation type (e.g., the co-author relations between publications in DBLP). Then we obtain embeddings through CLEH and those baselines on the processed HINs. For CLEH, we directly use the trained intermediate event representations generated by the event pooling (see (11)). For the baselines, we also apply the event pooling on the trained node embeddings to get event representations. The Hadamard product and the relation classification via a trained logistic regression classifier are sequentially executed. The average AUC and AP results of 10 repeated experiments are reported in Tables IX, X, and XI. In each column of the result tables, the largest value is in bold and the second largest value is underlined.

According to the introductions of our CLEH and the baselines, we have the following expectations. The evaluated methods would be divided into three grades based on the result values. In the first grade, CLEH fully sketches the generative process

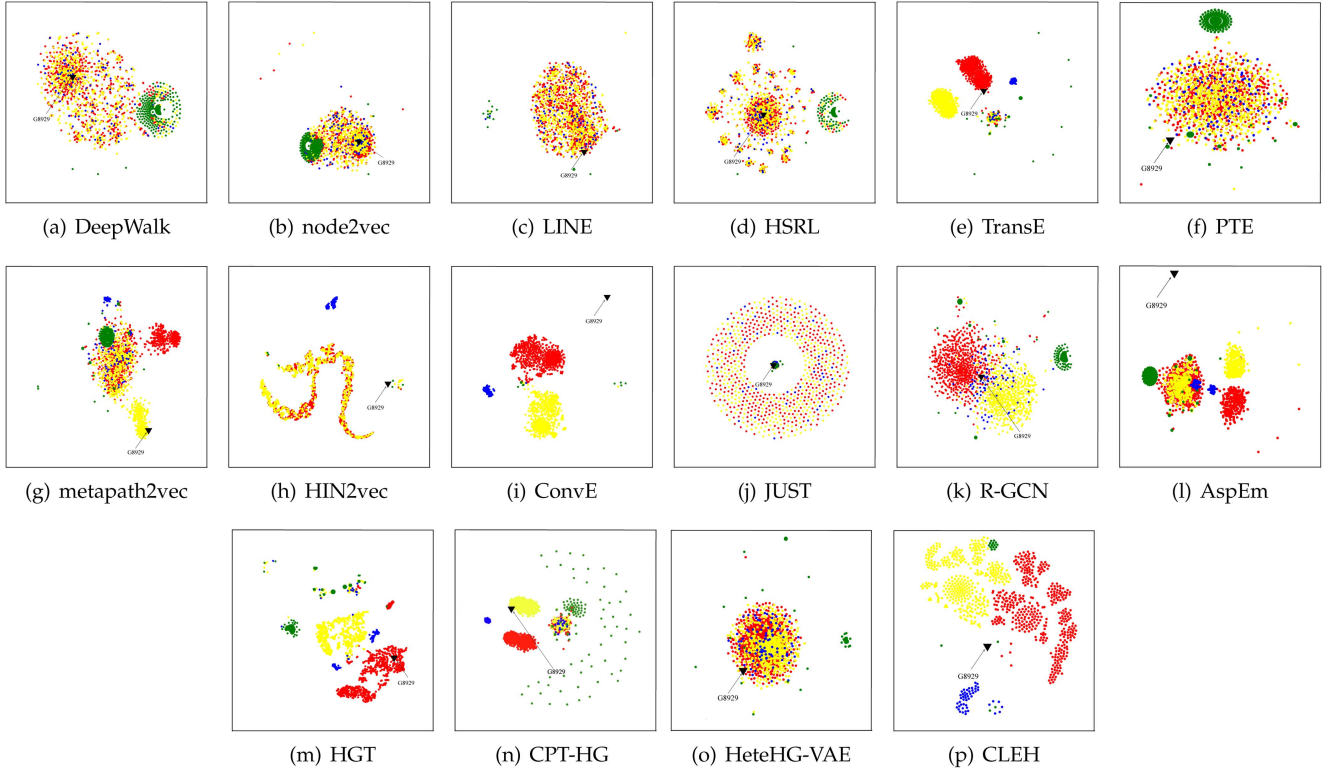


Fig. 5. Visualisation focusing on a central node G8929 (black), its type-specific neighbors (blue for species, red for chemicals and yellow for diseases) and its non-neighbors (green) in PubMed network.

TABLE XI
EVENT RELATION PREDICTION RESULTS ON YELP NETWORK WITH 20% OF
INTER-EVENT RELATIONS REMOVED

Relation Type	E-L-E		E-P-E		E-S-E	
	AUC	AP	AUC	AP	AUC	AP
DeepWalk	0.5342	0.5411	0.5230	0.5327	0.4925	0.4937
PTE	0.5024	0.5120	0.4988	0.4897	0.5122	0.5086
metapath2vec	0.7326	0.7442	0.7028	0.7149	0.7250	0.7308
HIN2vec	0.7423	0.7551	0.7250	0.7333	0.7451	0.7443
ConvE	0.7396	0.7452	0.7563	0.7440	0.7167	0.7285
JUST	0.4988	0.4753	0.4824	0.4930	0.5022	0.4827
R-GCN	0.5324	0.5510	0.5022	0.5397	0.5111	0.5286
AspEm	0.4082	0.4123	0.4650	0.4701	0.4999	0.5024
HGT	0.7344	0.7458	0.7543	0.7661	0.7489	0.7497
CPT-HG	0.7236	0.7345	0.7301	0.7452	0.7075	0.7114
HeteHG-VAE	0.7996	0.8057	0.8127	0.8225	0.7814	0.7923
CLEH	0.8442	0.8520	0.8673	0.8790	0.8484	0.8657

by which it would significantly outperform the other methods. In the second grade, HeteHG-VAE reconstructs only intra-event but no inter-event relations, leading to an expectation of lower performance compared to CLEH. In the third grade, the baselines excluding HeteHG-VAE do not consider the higher-order event semantics, anticipating lower performance compared to both CLEH and HeteHG-VAE. The experimental results completely align with our expectations.

F. Visualization

To gain better insights into the difference between CLEH and the baseline methods, we visualize the learned node embeddings via the t-SNE tool [65], which maps the inferred node embeddings into a 2D space.

We focus on a central node in the PubMed network, namely G8929. After mapping the embeddings of the target node, its different types of neighbors, and some randomly sampled non-neighbors into 2D space, we obtain the results shown in Fig. 5. The black triangle indicates the target central node. The yellow, red, and blue dots indicate its three types of neighbors: diseases, chemicals, and species, respectively. The green dots indicate the non-neighbors. In the ideal situation, the results should at least satisfy three conditions: (1) each type of neighbors should form a cluster, which shows whether a model can distinguish the node type; (2) the three types of neighbors' points should surround the target node equivalently as they are all directly connected to the target node; (3) the dots of neighbors should be closer to the target node than non-neighbors, which shows whether a model is competent for link prediction. As we can observe, most of the baselines only meet one of the three conditions. Despite the competitiveness of CPT-HG and HeteHG-VAE in quantitative experiments, their deficiencies are revealed through 2D visualization. HeteHG-VAE misses conditions (1) and (2) as it only considers the intra-event higher-order interactions. CPT-HG misses condition (1), performing better on typed link prediction than general link prediction. CLEH is the only one that meets all the three conditions. Achieving conditions (1) and

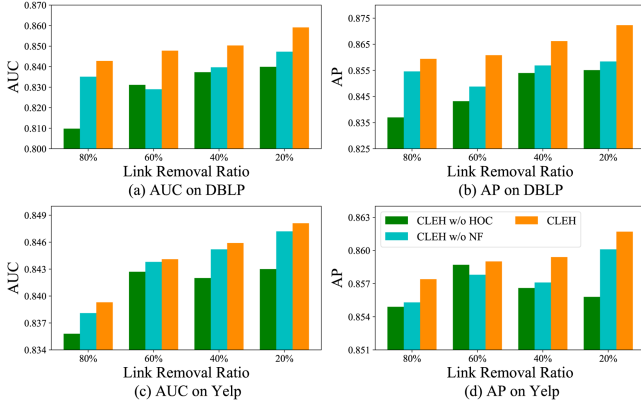


Fig. 6. Ablation results of CLEH on DBLP and Yelp.

(2) benefits from the way CLEH models from local topology to higher-order semantics, i.e., constructing the type-specific bipartite graphs. Achieving condition (3) is a result of higher-order contrastive learning. Thus, capturing the generative essence of HINs improves the quality of learned node embeddings.

G. Ablation Experiment

In order to verify the effectiveness of the normalizing-flow mechanism and higher-order contrastive learning, the following ablation experiments are conducted. We obtain two variants of CLEH by removing the flow component (CLEH w/o NF) and training by node-based link prediction instead of event-based relation prediction (CLEH w/o HOC), respectively. Subsequently, link prediction with them is implemented on DBLP and Yelp networks.

We illustrate the ablation results in Fig. 6. On both networks, CLEH consistently outperforms CLEH w/o NF and CLEH w/o HOC at all link removal ratios, as expected. The notably inferior performance of CLEH w/o HOC underscores the efficacy of our generative-process-sketching idea. The enhancement in the flow component on DBLP is more pronounced than on Yelp, mainly due to Yelp's relatively monotonous edge-type distribution. What's more, CLEH w/o NF also outperforms the baselines according to Tables III and V, as it captures the relations within and between events from a macro perspective.

H. Runtime Analysis

As we analysed, the cost time of CLEH is linear with the edge numbers $|E|$. Here we empirically verify this conclusion. We run our method and some of the most recent baselines on the HINs where some edges are removed. We show the training time per epoch on the three real-world HINs in Fig. 7. As the link removal ratio decreases, the training time per epoch increases linearly. CLEH is more efficient than HGT, HeteHG-VAE and CPT-HG.

I. Parameter Analysis

Here we study how the only two parameters of CLEH, i.e., the temperature parameter τ and embedding dimension d , affect the

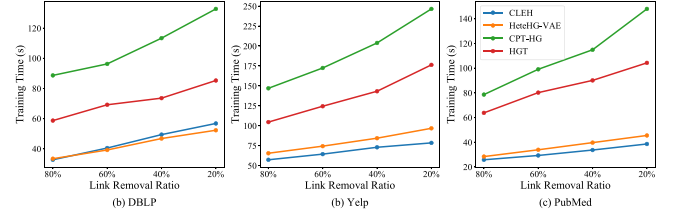


Fig. 7. Training time with different link removal ratios.

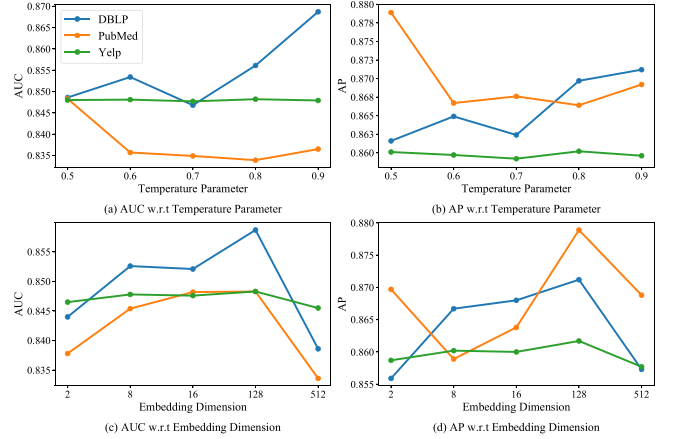


Fig. 8. Parameter sensitivity analysis.

performance. To this end, we train CLEH on all three networks with 20% of edges removed. We fix one parameter to the default value and tune the other one to find out how the results change. The results are demonstrated in Fig. 8.

1) *Temperature Parameter τ* : The test range for τ is $[0.5, 0.9]$. As shown in parts (a) and (b) of Fig. 8, the results on the three datasets show different patterns. With increasing τ , both AUC and AP improve on DBLP, decrease on PubMed, and remain almost constant on Yelp. This phenomenon can be attributed to the imbalance in the distribution of different types of links. According to (5), a smaller τ would enlarge the computed similarity between event embeddings. In DBLP where only one type of links is the major part, a large τ would alleviate the excessive gathering of events connected by them in the embedding space. In PubMed, where only one type of links is the minor part, the situation is reversed. As Yelp almost has only one edge type, changing τ on it has very little effect.

2) *Embedding Dimension d* : We raise d from 2 to 512 exponentially. As evident Fig. 8(c) and (d), we observe that both AUC and AP first increase and then decrease on all the networks. The peaks appear when $d = 128$. This phenomenon arises because embeddings need a sufficiently large dimension to contain more information but an excessively large dimension would lead to the overfitting problem.

VI. CONCLUSION AND FUTURE WORK

In this paper, we advocate the idea that sketching the generative process of HINs greatly helps node representation learning for link prediction. Given that existing HIN embedding

algorithms have not fully embraced it, we introduce CLEH as a method to implement and validate this innovative idea. CLEH consists of two essential modules: embedding generation and higher-order contrastive learning. The former involves constructing type-specific bipartite graphs and utilizing a flow-based encoder to learn node and event representations to capture intra-event relations. The latter focuses on learning inter-event relations through sampled event pairs. Together, these two parts concisely draw the bone of the whole HIN. Our CLEH is flexible as it has only two parameters.

To evaluate CLEH, we have conducted both qualitative and quantitative experiments on three real-world HIN datasets. A large set of embedding baselines are employed for comparison. Across tasks such as general and typed link prediction and event relation prediction, CLEH consistently exhibits outstanding performance. The visualization results intuitively show how sketching the generative process influence learned embeddings. The runtime analysis demonstrates the scalability of CLEH. What's more, we analyse how the normalizing-flow mechanism, higher-order contrastive learning, and two hyperparameters influence the prediction results.

Though we verify our idea via the proposed CLEH, there remains room for amelioration. Future work will explore more effective ways to sketch the generative process in HIN embedding models. The event-based higher-order contrastive learning is currently mechanical. We aim to explore and design a more effective and efficient approach for applying contrastive learning to events. In addition, building a genuine generative embedding model beyond just mimicking the generative process is considered.

REFERENCES

- [1] J. Gilmer, S. S. Schoenholz, P. F. Riley, O. Vinyals, and G. E. Dahl, "Neural message passing for quantum chemistry," in *Proc. 34th Int. Conf. Mach. Learn.*, PMLR, 2017, pp. 1263–1272.
- [2] C. Yang, L. Bai, C. Zhang, Q. Yuan, and J. Han, "Bridging collaborative filtering and semi-supervised learning: A neural approach for poi recommendation," in *Proc. 23rd ACM SIGKDD Int. Conf. Knowl. Discov. Data Mining*, 2017, pp. 1245–1254.
- [3] Y. Xiao, A. Krishnan, and H. Sundaram, "Discovering strategic behaviors for collaborative content-production in social networks," in *Proc. Web Conf.*, 2020, pp. 2078–2088.
- [4] C. Shi, Y. Li, J. Zhang, Y. Sun, and P. S. Yu, "A survey of heterogeneous information network analysis," *IEEE Trans. Knowl. Data Eng.*, vol. 29, no. 1, pp. 17–37, Jan. 2017.
- [5] Y. Fang, X. Zhao, P. Huang, W. Xiao, and M. de Rijke, "Scalable representation learning for dynamic heterogeneous information networks via metagraphs," *ACM Trans. Inf. Syst.*, vol. 40, no. 4, pp. 1–27, 2022.
- [6] C. Yang, Y. Xiao, Y. Zhang, Y. Sun, and J. Han, "Heterogeneous network representation learning: A unified framework with survey and benchmark," *IEEE Trans. Knowl. Data Eng.*, vol. 34, no. 10, pp. 4854–4873, Oct. 2022.
- [7] Y. Dong, Z. Hu, K. Wang, Y. Sun, and J. Tang, "Heterogeneous network representation learning," in *Proc. 29th Int. Joint Conf. Artif. Intell.*, 2020, pp. 4861–4867.
- [8] H. Peng, R. Zhang, Y. Dou, R. Yang, J. Zhang, and P. S. Yu, "Reinforced neighborhood selection guided multi-relational graph neural networks," *ACM Trans. Inf. Syst.*, vol. 40, no. 4, pp. 1–46, 2021.
- [9] Y. Dong, N. V. Chawla, and A. Swami, "Metapath2Vec: Scalable representation learning for heterogeneous networks," in *Proc. 23rd ACM SIGKDD Int. Conf. Knowl. Discov. Data Mining*, 2017, pp. 135–144.
- [10] T.-Y. Fu, W.-C. Lee, and Z. Lei, "Hin2Vec: Explore meta-paths in heterogeneous information networks for representation learning," in *Proc. 2017 ACM Conf. Inf. Knowl. Manage.*, 2017, pp. 1797–1806.
- [11] R. Hussein, D. Yang, and P. Cudré-Mauroux, "Are meta-paths necessary? Revisiting heterogeneous graph embeddings," in *Proc. 27th ACM Int. Conf. Inf. Knowl. Manage.*, 2018, pp. 437–446.
- [12] Y. Shi, Q. Zhu, F. Guo, C. Zhang, and J. Han, "Easing embedding learning by comprehensive transcription of heterogeneous information networks," in *Proc. 24th ACM SIGKDD Int. Conf. Knowl. Discov. Data Mining*, 2018, pp. 2190–2199.
- [13] H. Hong, H. Guo, Y. Lin, X. Yang, Z. Li, and J. Ye, "An attention-based graph neural network for heterogeneous structural learning," in *Proc. 34th AAAI Conf. Artif. Intell.*, 2020, pp. 4132–4139.
- [14] Z. Hu, Y. Dong, K. Wang, and Y. Sun, "Heterogeneous graph transformer," in *Proc. Web Conf.*, 2020, pp. 2704–2710.
- [15] Q. Long, L. Xu, Z. Fang, and G. Song, "HGK-GNN: Heterogeneous graph kernel based graph neural networks," in *Proc. 27th ACM SIGKDD Conf. Knowl. Discov. Data Mining*, 2021, pp. 1129–1138.
- [16] M. Schlichtkrull, T. N. Kipf, P. Bloem, R. Van Den Berg, I. Titov, and M. Welling, "Modeling relational data with graph convolutional networks," in *Proc. Eur. Semantic Web Conf.*, Springer, 2018, pp. 593–607.
- [17] A. Bordes, N. Usunier, A. Garcia-Duran, J. Weston, and O. Yakhnenko, "Translating embeddings for modeling multi-relational data," in *Proc. Adv. Neural Inf. Process. Syst.*, 2013, pp. 2787–2795.
- [18] T. Dettmers, P. Minervini, P. Stenetorp, and S. Riedel, "Convolutional 2D knowledge graph embeddings," in *Proc. 32nd AAAI Conf. Artif. Intell.*, 2018, pp. 1811–1818.
- [19] Y. Lu, C. Shi, L. Hu, and Z. Liu, "Relation structure-aware heterogeneous information network embedding," in *Proc. 33rd AAAI Conf. Artif. Intell.*, 2019, pp. 4456–4463.
- [20] X. Wang et al., "Heterogeneous graph attention network," in *Proc. World Wide Web Conf.*, 2019, pp. 2022–2032.
- [21] Y. Zhang, R. He, Z. Liu, K. H. Lim, and L. Bing, "An unsupervised sentence embedding method by mutual information maximization," in *Proc. 2020 Conf. Empirical Methods Natural Lang. Process.*, 2020, pp. 1601–1610.
- [22] T. Gao, X. Yao, and D. Chen, "SimCSE: Simple contrastive learning of sentence embeddings," in *Proc. 2021 Conf. Empirical Methods Natural Lang. Process.*, 2021, pp. 6894–6910.
- [23] K. He, H. Fan, Y. Wu, S. Xie, and R. Girshick, "Momentum contrast for unsupervised visual representation learning," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit.*, 2020, pp. 9729–9738.
- [24] J. Zbontar, L. Jing, I. Misra, Y. LeCun, and S. Deny, "Barlow twins: Self-supervised learning via redundancy reduction," in *Proc. 38th Int. Conf. Mach. Learn.*, PMLR, 2021, pp. 12310–12320.
- [25] Y. Ren, B. Liu, C. Huang, P. Dai, L. Bo, and J. Zhang, "Heterogeneous deep graph infomax," 2019, *arXiv: 1911.08538*.
- [26] J. Qiu et al., "GCC: Graph contrastive coding for graph neural network pre-training," in *Proc. 26th ACM SIGKDD Int. Conf. Knowl. Discov. Data Mining*, 2020, pp. 1150–1160.
- [27] P. Velickovic, W. Fedus, L. W. Hamilton, P. Liò, Y. Bengio, and D. R. Hjelm, "Deep graph infomax," in *Proc. 7th Int. Conf. Learn. Representations*, 2019, Art. no. 4.
- [28] Y. You, T. Chen, Y. Sui, T. Chen, Z. Wang, and Y. Shen, "Graph contrastive learning with augmentations," in *Proc. Adv. Neural Inf. Process. Syst.*, 2020, pp. 5812–5823.
- [29] S. Suresh, P. Li, C. Hao, and J. Neville, "Adversarial graph augmentation to improve graph contrastive learning," in *Proc. Adv. Neural Inf. Process. Syst.*, 2021, pp. 15920–15933.
- [30] D. Xu, W. Cheng, D. Luo, H. Chen, and X. Zhang, "InfoGCL: Information-aware graph contrastive learning," in *Proc. Adv. Neural Inf. Process. Syst.*, 2021, pp. 30414–30425.
- [31] X. Wang, N. Liu, H. Han, and C. Shi, "Self-supervised heterogeneous graph neural network with co-contrastive learning," in *Proc. 27th ACM SIGKDD Conf. Knowl. Discov. Data Mining*, 2021, pp. 1726–1736.
- [32] D. He et al., "Analyzing heterogeneous networks with missing attributes by unsupervised contrastive learning," *IEEE Trans. Neural Netw. Learn. Syst.*, vol. 35, no. 4, pp. 4438–4450, Apr. 2024.
- [33] X. Jiang, Y. Lu, Y. Fang, and C. Shi, "Contrastive pre-training of GNNs on heterogeneous graphs," in *Proc. 30th ACM Int. Conf. Inf. Knowl. Manage.*, 2021, pp. 803–812.
- [34] Y. Dong et al., "Link prediction and recommendation across heterogeneous social networks," in *Proc. IEEE 12th Int. Conf. Data Mining*, 2012, pp. 181–190.
- [35] H. Chen, H. Yin, W. Wang, H. Wang, Q. V. H. Nguyen, and X. Li, "PME: Projected metric embedding on heterogeneous networks for link prediction," in *Proc. 24th ACM SIGKDD Int. Conf. Knowl. Discov. Data Mining*, 2018, pp. 1177–1186.

- [36] H. Fan et al., "Heterogeneous hypergraph variational autoencoder for link prediction," *IEEE Trans. Pattern Anal. Mach. Intell.*, vol. 44, no. 8, pp. 4125–4138, Aug. 2022.
- [37] D. P. Kingma and M. Welling, "Auto-encoding variational bayes," in *Proc. 2nd Int. Conf. Learn. Representations*, 2014.
- [38] J. Tang, M. Qu, and Q. Mei, "Pte: Predictive text embedding through large-scale heterogeneous text networks," in *Proc. 21th ACM SIGKDD Int. Conf. Knowl. Discov. Data Mining*, 2015, pp. 1165–1174.
- [39] H. Wang, F. Zhang, M. Hou, X. Xie, M. Guo, and Q. Liu, "Shine: Signed heterogeneous information network embedding for sentiment link prediction," in *Proc. 11th ACM Int. Conf. Web Search Data Mining*, 2018, pp. 592–600.
- [40] S. Vashishth, S. Sanyal, V. Nitin, and P. Talukdar, "Composition-based multi-relational graph convolutional networks," in *Proc. 8th Int. Conf. Learn. Representations*, 2020, pp. 1–14.
- [41] S. Ji, S. Pan, E. Cambria, P. Marttinen, and S. Y. Philip, "A survey on knowledge graphs: Representation, acquisition, and applications," *IEEE Trans. Neural Netw. Learn. Syst.*, vol. 33, no. 2, pp. 494–514, Feb. 2022.
- [42] A. Grover and J. Leskovec, "Node2vec: Scalable feature learning for networks," in *Proc. 22nd ACM SIGKDD Int. Conf. Knowl. Discov. Data Mining*, 2016, pp. 855–864.
- [43] Z. Meng, S. Liang, X. Zhang, R. McCreadie, and I. Ounis, "Jointly learning representations of nodes and attributes for attributed networks," *ACM Trans. Inf. Syst.*, vol. 38, no. 2, pp. 1–32, 2020.
- [44] T. N. Kipf and M. Welling, "Variational graph auto-encoders," in *Proc. NIPS Workshop Bayesian Deep Learn.*, 2016, Art. no. 21.
- [45] Y. Yang, N. Chawla, Y. Sun, and J. Han, "Predicting links in multi-relational and heterogeneous networks," in *Proc. IEEE 12th Int. Conf. Data Mining*, 2012, pp. 755–764.
- [46] P. Wang, K. Agarwal, C. Ham, S. Choudhury, and C. K. Reddy, "Self-supervised learning of contextual embeddings for link prediction in heterogeneous networks," in *Proc. ACM Web Conf.*, 2021, pp. 2946–2957.
- [47] K. Hassani and A. H. Khasahmadi, "Contrastive multi-view representation learning on graphs," in *Proc. 37th Int. Conf. Mach. Learn.*, PMLR, 2020, pp. 4116–4126.
- [48] R. H. Devos et al., "Learning deep representations by mutual information estimation and maximization," in *Proc. 7th Int. Conf. Learn. Representations*, 2019, pp. 1–14.
- [49] F. Battiston et al., "The physics of higher-order interactions in complex systems," *Nat. Phys.*, vol. 17, no. 10, pp. 1093–1098, 2021.
- [50] H. Gui et al., "Embedding learning with events in heterogeneous information networks," *IEEE Trans. Knowl. Data Eng.*, vol. 29, no. 11, pp. 2428–2441, Nov. 2017.
- [51] M. U. Gutmann and A. Hyvärinen, "Noise-contrastive estimation of unnormalized statistical models, with applications to natural image statistics," *J. Mach. Learn. Res.*, vol. 13, no. 2, pp. 307–361, 2012.
- [52] A. v. d. Oord, Y. Li, and O. Vinyals, "Representation learning with contrastive predictive coding," 2018, *arXiv: 1807.03748*.
- [53] K. Nozawa and I. Sato, "Understanding negative samples in instance discriminative self-supervised representation learning," in *Proc. Adv. Neural Inf. Process. Syst.*, 2021, pp. 5784–5797.
- [54] C. Wu, F. Wu, and Y. Huang, "Rethinking infonce: How many negative samples do you need?," in *Proc. 31st Int. Joint Conf. Artif. Intell.*, 2022, pp. 2509–2515.
- [55] H. Shan, D. Jin, P. Jiao, Z. Liu, B. Li, and Y. Huang, "NF-VGA: Incorporating normalizing flows into graph variational autoencoder for embedding attribute networks," in *Proc. 2020 IEEE Int. Conf. Data Mining*, 2020, pp. 1244–1249.
- [56] L. Dinh, J. Sohl-Dickstein, and S. Bengio, "Density estimation using real NVP," in *Proc. 5th Int. Conf. Learn. Representations*, 2017, pp. 1–12.
- [57] K. Xu, W. Hu, J. Leskovec, and S. Jegelka, "How powerful are graph neural networks?," in *Proc. 7th Int. Conf. Learn. Representations*, 2019, pp. 1–13.
- [58] B. Perozzi, R. Al-Rfou, and S. Skiena, "Deepwalk: Online learning of social representations," in *Proc. 20th ACM SIGKDD Int. Conf. Knowl. Discov. Data Mining*, 2014, pp. 701–710.
- [59] J. Tang, M. Qu, M. Wang, M. Zhang, J. Yan, and Q. Mei, "Line: Large-scale information network embedding," in *Proc. 24th Int. Conf. World Wide Web*, 2015, pp. 1067–1077.
- [60] G. Fu, C. Hou, and X. Yao, "Learning topological representation for networks via hierarchical sampling," in *Proc. IEEE 2019 Int. Joint Conf. Neural Netw.*, 2019, pp. 1–8.
- [61] N. T. Kipf and M. Welling, "Semi-supervised classification with graph convolutional networks," in *Proc. 5th Int. Conf. Learn. Representations*, 2017, pp. 1–10.
- [62] Y. Shi, H. Gui, Q. Zhu, L. Kaplan, and J. Han, "AspEm: Embedding learning by aspects in heterogeneous information networks," in *Proc. 2018 SIAM Int. Conf. Data Mining*, 2018, pp. 144–152.
- [63] P. D. Kingma and L. J. Ba, "Adam: A method for stochastic optimization," in *Proc. 3rd Int. Conf. Learn. Representations*, 2015, pp. 1–11.
- [64] M. Zhang and Y. Chen, "Link prediction based on graph neural networks," in *Proc. Adv. Neural Inf. Process. Syst.*, 2018, pp. 5165–5175.
- [65] L. Van der Maaten and G. Hinton, "Visualizing data using t-SNE," *J. Mach. Learn. Res.*, vol. 9, no. 11, pp. 2579–2605, 2008.



Xuan Guo is currently working toward the doctoral degree with the College of Intelligence and Computing, Tianjin University, Tianjin, China. His current research interests include complex network analysis, heterogeneous information network, network representation learning, and role discovery.



Pengfei Jiao (Member, IEEE) received the PhD degrees in computer science from Tianjin University, Tianjin, China, in 2018. From 2018 to 2021, he was a lecture with the Center of Biosafety Research and Strategy of Tianjin University. He is currently a professor with the School of Cyberspace, Hangzhou Dianzi University, Hangzhou, China. His current research interests include complex network analysis and its applications.



Danyang Shi received the bachelor's degree from Tianjin University in 2020. He is currently working toward the master's degree with the College of Intelligence and Computing, Tianjin University. His current research interest is heterogeneous information network representation learning.



Jie Li received the bachelor's degree from the Taiyuan University of Technology in 2022. She is currently working toward the master's degree with the College of Intelligence and Computing, Tianjin University. Her current research interest is heterogeneous network representation learning.



Jinpeng Wang received the PhD degree in computer science from l'Université Clermont-Auvergne, Clermont, France, in 2019. From 2019 to 2021, he did research in Zhejiang Lab. Now he is currently an assistant researcher with the School of Cyberspace, Hangzhou Dianzi University, Hangzhou, China. His current research interests include swarm intelligence, wireless network and its applications.